중간보고서 v5 검토 메모 — 파트별 논리 점검과 보강 실험·설계 제안

작성 2026-10-08. 대상: 중간보고서_이선재_v5.pdf(10 월 2 일~8 일 작업을 반영한 개정판). 각 파트에 대해 (1) 보고서가 현재 세우고 있는 논리 사슬, (2) 심사자가 파고들 수 있는 약점, (3) 그 약점을 메우는 실험·설계를 적는다. 제안은 GPU 가 필요한지, 예상 소요, 무엇을 입증하는지를 함께 적어 우선순위를 가릴 수 있게 했다.

0. 요약: 우선순위 상위 6 개

순위	제안	파트	GPU	소요	입증하는 것
1	**정적 최선 분할 수(예: 항상 16) 기준선 추가**	평가	격자는 불필요, 실생성은 필요	0.5 일	"왜 동적 선택이어야 하는가" — 고정 설정 하나로 얻는 이득과의 차이
2	**트레이스 구동 end-to-end 측정** (Azure 트레이스 일부를 엔진으로 재생)	평가	필요	1 일	빈도 분석(4 장 다)과 실생성 이득(4 장 라)을 같은 부하에서 연결
3	**트레이스 채점의 실측 검증**(예측 손실 구간별로 표본 셀 실측)	문제 정의	필요	0.5 일	13.5%라는 숫자가 모델 예측이 아니라 실측으로 뒷받침됨
4	**자연어 벤치마크에서 출력 품질 비교** (예: 수백 문항 greedy 정확도)	평가	필요	0.5 일	출력 동일성 주장을 합성 프롬프트 밖에서 확인
5	**vLLM 디코드 단계 프로파일링**	디자인·평가	필요	1 일	운영 엔진에서 본 작품의 기전이 성립하는 조건·불성립 원인
6	**다음 단계 결정의 선계산**(길이가 매 단계 1 씩 늘어나는 성질 이용)	디자인	필요	1 일	첫 결정 비용을 임계 경로에서 제거 → 하이브리드·모델 정책의 실용성

1. 문제 정의와 연구 배경 (1 장, 4 장 나·다)

1.1 현재의 논리 사슬

- 디코드 attention 은 memory-bound 이고 병렬성은 배치×헤드 수로 정해진다(Flash-Decoding).
- FlashAttention-2 의 분할 수 휴리스틱은 $B \cdot H_{kv} \cdot m \geq 0.8 \times (2 \times SM)$이면 길이를 보지 않고 분할하지 않는다. FlashInfer 의 기본 변형 선택도 길이를 보지 않는다.
- 따라서 가변 길이 배치에서 부하 불균형이 생긴다 — 커널 격자에서 중앙값 1.44 배, 최악 3.98 배(페이지 KV), 64K 에서 최악 5.18 배.
- 이 조건은 실제 트래픽에서 단계의 0.3~14%에 해당한다(트레이스 재생).
- 그러므로 길이 분포를 보고 설정을 고르는 선택기가 필요하다.

사슬 자체는 닫혀 있다. 약점은 4 번의 근거 강도와, 2 번의 일반성이다.

1.2 약점

- **W1. 제목과 본문의 불일치.** 제목은 "시뮬레이션 기반 … 동적 선택"이지만, 선택은 전부 실측 테이블과 실측으로 맞춘 성능 모델에서 나오고 Accel-Sim 은 보정되지 않아(1.05~3.06 배) 선택에 기여하지 않는다. 본문은 이미 "시뮬레이터는 원인 설명·민감도 분석용"이라고 적고 있으나, 제목만 읽는 심사자는 시뮬레이션 기반 선택을 기대한다. 제목은 신청서에 등록된 것이라 바꾸기 어렵다면, 요약 첫 문단과 1 장 나)에서 "시뮬레이션은 선택이 아니라 설명에 쓴다"를 한 문장으로 선언하는 편이 안전하다(현재는 2 장 다)와 PLAN 에만 있다).
- **W2. 빈도 수치(13.5%)는 성능 모델의 예측이며 부하 배율에 민감하다.** ×0.5 에서 2.4%, ×1 에서 13.5%, ×2 에서 4.1%로 봉우리 모양이다. "GPU 한 장의 부하 = 도착 간격 원본"이라는 선택이 임의적이라는 지적이 가능하다. 또 재생기는 단계 시간 30 ms 고정·즉시 프리필·FIFO·배치 상한 64 를 가정한다.
- **W3. 어느 트레이스에도 32K 이상 요청이 없다.** 보고서의 대표 조건(32K×1+512×31, 64K)은 트레이스에 나타나지 않는 극단이고, 긴 문맥 조건은 프롬프트 ×2·×4 가상 실험으로만 뒷받침된다.
- **W4. "배치 26 이상"은 헤드 구성 32/8·SM 128 개에 한정된 수치다.** 일반식은 $B \cdot H_{kv} \cdot m \geq 0.8 \times 2 \times SM$ 이므로 H_{kv} 가 4 면 $B \geq 52$, 16 이면 $B \geq 13$ 이다. 모델·GPU 가 바뀌면 조건이 바뀐다는 점을 표로 보이지 않으면 결과가 Qwen3-4B 특유로 읽힐 수 있다.
- **W5. FlashInfer 기본 변형에 대한 서술의 근거.** "GQA 그룹 크기 ⩾ 4 면 tensor-core 권장"은 FlashInfer 문서의 권장이고, vLLM 이 그 값을 쓴다는 것은 검토 문서의 조사 결과다. 버전(flashinfer-python 0.6.13)과 그 근거를 각주 수준으로라도 적어야 "두 라이브러리의 기본 설정이 틀린다"는 확장된 정의가 서 있다.

1.3 보강 실험·설계

- **E1-1. 트레이스 채점의 실측 검증(GPU, 30~60 분).** 재생된 단계의 길이 벡터를 예측 손실 구간(<1.1, 1.1~1.25, 1.25~1.5, >1.5)별로 층화 표본 추출(각 20~30 개)하여 4090 에서 휴리스틱과 최선 분할 수의 커널 시간을 실측한다. 보고할 것은 "예측 ⩾1.25 배 중 실측도 ⩾1.25 배인 비율"(precision)과 구간별 예측 오차. 이것으로 4 장 다)의 수치가 "모델 예측"에서 "실측으로 검증된 예측"이 된다. 기존 kernelscope bench 에 셀 목록만 넣으면 되므로 구현 비용이 작다.
- **E1-2. 부하 스윕(GPU 불필요, 1~2 시간).** 도착 배율 0.25~4 를 8~10 단계로 재생하여 "평균 배치 크기 대 손실 단계 비율" 곡선을 그린다. 부하의 정의를 배율이 아니라 평균 큐 대기 시간이나 GPU 점유율로 바꾸어 "SLO 를 만족하는 최대 부하 근처에서 몇 %인가"로 보고하면 ×1 이 임의라는 지적을 피할 수 있다.
- **E1-3. 긴 문맥 트레이스 재생(GPU 불필요, 1~2 시간).** Mooncake(Kimi) 공개 트레이스처럼 평균 입력 길이가 수천~수만 토큰인 트레이스를 같은 재생기로 돌린다. 이것이 있으면 64K 격자와 what-if ×4 가 "가상"이 아니라 "실제 서비스에 존재하는 분포"에 연결된다.
- **E1-4. 헤드 구성·GPU 별 임계 표(GPU 불필요, 1 시간).** 공개 모델 6~8 종(Llama-3-8B/70B, Qwen3-4B/32B, Mistral, Gemma 등)의 H_{kv} 와 SM 수(4090/A100/H100)로 "분할 없음 임계 배치 크기"를 표로 만든다. 이미 측정한 S2 헤드 구성 격자(135 셀)가 이 표의 실측 검증점이 된다.
- **설계 서술 보완.** 1 장 (2)에 FlashAttention-3(Hopper)는 디코드 경로와 휴리스틱이 다르므로 본 작품의 주장은 "FA2 2.8.x + SM 128 개급 Ada"에 한정된다는 범위 문장을 넣는다.

2. 관련 연구 (2 장)

2.1 현재의 구성

서빙 시스템 (Orca·vLLM·Sarathi-Serve·트레이스), attention 커널 (FA·FA2·Flash-Decoding·FlashDecoding++·FlashInfer·LeanAttention), GPU 분석·시뮬레이션(Roofline·GPGPU-Sim·Accel-Sim·Vidur), 원안과의 관계. 커널 수준 해법(FlashInfer·LeanAttention)을 "상한"으로 두고 그 격차를 같은 엔진에서 측정했다는 위치 잡기는 적절하다.

2.2 약점

- **W1. 가장 가까운 방법론 계열이 빠져 있다.** 본 작품의 방법은 "입력 모양에 따른 커널 설정의 런타임 선택 + 실측 테이블/모델 + regret 평가"인데, 이는 커널 자동 튜닝(TVM/Ansor, Triton autotune, cuBLAS/cuDNN 휴리스틱)과 추론 엔진의 커널 디스패치(TensorRT-LLM 의 XQA 커널 선택, SGLang 의 attention 백엔드 선택)와 같은 계열이다. 이 계열을 한 문단으로 정리하고 "오프라인 모양별 튜닝"과 "매 디코드 단계의 길이 분포 기반 선택"의 차이를 적어야 기여가 선다.
- **W2. 출력 동일성 분석의 문헌 근거가 없다.** 분할 수에 따른 합산 순서 차이로 로짓이 흔들리고 greedy 경로가 갈린다는 분석은 "LLM 추론의 비결정성/배치 불변성"(batch invariance) 논의와 정확히 같은 문제다(예: Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference", 2025 — split-KV 의 reduction 순서를 배치 불변으로 고정하는 방법을 다룬다). 이를 인용하면 4 장 라)(5)의 분류·커널 검사 방법이 임의의 고안이 아니라 알려진 현상에 대한 표준적 대응임을 보일 수 있고, "배치 불변 커널을 쓰면 분기 자체가 사라진다"는 대안과의 관계도 정리된다.
- **W3. FlashAttention-3·최신 FlashInfer 와의 관계.** Hopper 에서는 FA3 가 쓰이고 FlashInfer 도 버전에 따라 변형 선택 로직이 바뀔 수 있다. "본 작품의 관찰이 어느 버전·어느 세대까지 유효한가"를 관련 연구에서 한 문장으로 한정해야 한다.
- **W4. LeanAttention 서술의 정확성.** "stream-K 방식의 reduction"이라는 요약은 논문 초록과 일치하지만, 본 작품이 직접 측정하지 않았으므로 "보고하였다" 수준의 서술로 두고 비교 대상에서 제외됨을 명시하는 편이 안전하다(현재 그렇게 되어 있음).

2.3 보강

- 2 장 나)에 **커널 자동 튜닝·런타임 디스패치 문단** 추가(인용 3~4 건).
- 2 장에 **수치 비결정성·배치 불변성 문단** 추가(인용 1~2 건) 후 4 장 라)(5)에서 재인용.
- 표 2 에 "커널 자동 튜닝" 행과 "배치 불변 커널" 행을 추가하여 본 작품과의 관계(각각 "오프라인 vs 런타임", "분기를 없애는 대안 vs 분기를 측정·분류")를 적는다.

3. 디자인 (3 장)

3.1 현재의 구성

커널 측정 하네스 → 실측 기반 성능 모델 → 정책(휴리스틱·고정·테이블·모델·하이브리드·FlashInfer 백엔드·라이브러리 교체 테이블) → 실행 엔진(FA2/FlashInfer, 페이지 KV, 정책 캐시) → 출력 검증(분기 지점·ULP 분류·커널 수준 검사) → 검증 531 항목·대시보드.

3.2 약점

- **W1. 조회 테이블의 특징 공간이 정당화되지 않았다.** 최근접 셀 탐색의 특징은 log2(배치 크기, 최대 길이, 평균 길이) 3 개다. 이 특징은 "긴 요청 2 개+짧은 30 개"와 "두 배 긴 요청 1 개+짧은 31 개"를 구분하지 못할 수 있고, 특징 선택이나 거리 척도의 대안(표준편차, 긴 요청 개수, CTA 작업량 벡터의 거리)을 비교한 결과가 없다. leave-one-out 평가에서 테이블 단독 최악 손실이 50~100%였다는 사실(STATUS 9 월 26 일)이 이 약점의 증거다.
- **W2. 하이브리드 정책의 δ=0.2 는 민감도 분석 없이 고정되었다.** δ 를 0.05~0.5 로 바꿀 때 leave-one-out 최악 손실 곡선이 없으면 "검증 집합에 맞춘 값"이라는 지적을 받는다.
- **W3. 성능 모델의 잔차 분석이 없다.** 모델 단독 최악 손실 24~66%가 어떤 셀(큰 배치? 긴 요청 2 개? warm?)에서 나는지, 순위 역전이 어느 후보 쌍 사이에서 나는지 분석이 없어 모델 개선 방향이 서지 않는다. warm V1 에서 모든 정책이 41.7%로 실패하는 것도 "warm 대역폭 항"이라고만 적혀 있다.
- **W4. 첫 결정 비용이 설계상 임계 경로에 있다.** 모델·하이브리드 정책의 캐시 미스 1 회는 14~19 ms 로 디코드 단계(25 ms)와 같은 자릿수다. 캐시 유지 규약으로 측정한 결과는 "같은 구성이 되풀이되는 부하"에서의 낙관적 경계다. 그런데 디코드에서는 매 단계 모든 요청의 길이가 정확히 1 씩 늘어나므로, 다음 페이지 경계에서 캐시 키가 어떻게 바뀔지 **결정적으로 예측 가능**하다. 즉 다음 결정을 현재 단계가 실행되는 동안 배경 스레드에서 미리 계산할 수 있는데, 이 설계가 빠져 있다.
- **W5. FlashInfer plan 비용의 상각.** 라이브러리 교체 테이블의 손익을 가른 것이 단계당 plan 370 μs 인데, plan 을 단계 사이에 재사용하거나 CUDA Graph 모드로 상각할 수 있는지 조사하지 않았다(보고서가 한계로 명시함). 이 조사 없이는 "라이브러리 교체의 추가 이득은 약 1%"라는 결론이 "본 작품의 통합 방식에서"라는 단서를 달아야 한다.
- **W6. 자체 엔진에서만 검증되었고 운영 엔진으로의 이식 경로가 없다.** vLLM 에서는 기전이 확인되지 않았다. 정책이 vLLM 의 attention 백엔드에 어떻게 꽂히는지(num_splits 를 단계마다 설정하는 패치의 실현 가능성)가 설계에 없다.
- **W7. 출력 검증의 문턱(2 ULP)과 검사 범위.** 1·2 ULP 문턱은 합리적이지만 근거 서술이 없고, 커널 수준 검사는 사건이 난 한 단계에서만 수행된다. 분기가 없는 조건에서도 커널이 같은 입력에 같은 출력을 내는지(배치 불변성·분할 불변성)를 체계적으로 검사하는 절차가 없다.

3.3 보강 실험·설계

- **D1. 테이블 특징·거리 ablation(GPU 불필요, 반나절).** 기존 leave-one-out 평가기(scripts/evaluate_hybrid.py)에 특징 집합 3~4 종(현행 / +표준편차 / +긴 요청 수 / CTA 작업량 벡터 거리)을 넣어 V1·V5·V6 의 최악·p90 손실을 표로 만든다. 현행이 최선이면 그대로 정당화되고, 아니면 테이블·하이브리드 정책이 함께 개선된다.
- **D2. δ 민감도 곡선(GPU 불필요, 1 시간).** $\delta \in \{0.05, 0.1, 0.15, 0.2, 0.3, 0.5\}$에 대해 세 집합의 최악 손실과 "모델을 따른 비율"을 그린다. 0.2 가 평탄한 구간에 있으면 그 자체가 정당화다.
- **D3. 성능 모델 잔차 분석(GPU 불필요, 반나절).** 손실 상위 5% 셀의 공통점(배치 크기, 긴 요청 수, 후보 쌍)을 정리하고, warm 실패의 원인을 L2 재사용 항의 부재로 특정할 수 있는지 본다. 결과는 모델 개선 또는 "모델을 쓰지 말아야 할 영역"의 명시로 이어진다.
- **D4. 다음 단계 결정 선계산(GPU, 1 일).** 정책 인터페이스에 prefetch(next_lens)를 추가하고, 엔진이 현재 단계의 커널을 띄운 뒤 다음 단계의 양자화 키를 미리 계산해 캐시를 채운다. 측정은 자연어 요청 도착

조건에서 캐시 초기화 규약 그대로 재는 것으로, 1.02 배(현행 fresh)가 1.09 배(keep)에 근접하면 성공이다. 하이브리드·모델 정책의 실용성에 관한 가장 큰 반론을 없앤다.

- **D5. plan 상각 조사(GPU, 반나절).** FlashInfer 디코드 래퍼의 CUDA Graph 모드(고정 버퍼에서 길이만 갱신)로 매 단계 plan 없이 동작하는지, 동작하면 단계당 비용이 얼마인지 잰다. 가능하면 라이브러리 교체 테이블을 재측정하고, 불가능하면 그 사실을 한계의 근거로 적는다.
- **D6. 운영 엔진 이식 경로(설계 문서, GPU 1 일).** vLLM FlashAttention 백엔드에서 num_splits 인자가 지나가는 경로를 확인하고, 정책을 꽂는 최소 패치의 설계를 적는다. 5 번(vLLM 프로파일링)이 "attention 이 단계 시간의 몇 %인가"를 먼저 답해야 이 이식이 의미 있는지 결정된다.
- **D7. 불변성 검사 절차(GPU, 반나절).** 같은 요청을 (a) 단독, (b) 가변 길이 배치 안, (c) 분할 수 1·8·16 으로 실행했을 때 로짓이 비트 단위로 같은 위치의 비율을 보고하는 검사를 추가한다 . 이것은 2 장에서 인용할 배치 불변성 문헌의 표준 검사와 같은 형식이라 심사자가 바로 이해한다.

4. 평가 (4 장)

4.1 현재의 구성

커널 격자(7,443 개 조건) → 트레이스 재생(20 개 구성) → 실생성(Qwen3-4B 3 개 시나리오 × 정책, 3~5 회; 64K; 후속 108 회) → 연산 클래스 분해 → FlashInfer 통합 비교 → 분기 분류·커널 검사 → 모델 오프라인 검증 → vLLM 재현 → 검증 531 항목.

4.2 약점

- **W1. "왜 동적 선택인가"에 답하는 기준선이 없다.** 가장 단순한 대안은 "항상 분할 수 16"(또는 vLLM CUDA Graph 모드의 상한 16)이다. 고정 분할 8 은 가변 길이에서 33.39 ms 로 테이블(33.80 ms)과 같은 수준이었고, 문제는 균일 길이에서의 출력 불일치였다. 그런데 균일 길이에서 고정 16 의 **TPOT** 이 얼마나 나빠지는지는 보고되지 않았다. 균일 격자에서 고정 16 의 손실이 몇 % 수준이면, 심사자는 "동적 선택 대신 항상 16 을 쓰면 되지 않는가"라고 물을 것이고 현재 보고서에는 답이 없다. 출력 불일치는 테이블 정책도 64K 에서 겪으므로 고정 설정을 배제하는 근거가 되지 못한다.
- **W2. 대표 수치(1.81 배, 2.42 배)는 테이블 정책이 측정 격자의 셀 위에서 낸 in-sample 결과다.** leave-one-out 에서 테이블 단독은 50~100%의 최악 손실을 보였다. 격자 밖 조건의 실측 이득은 자연어 held-out(1.09~1.29 배)뿐이다. 표 8 위 또는 아래에 "32K·64K 조건은 측정 격자와 같은 구성이며 격자 밖 일반화는 (3)과 108 회 후속 실험이 답한다"는 문장이 필요하다.
- **W3. 통계 보고.** 반복 3 회 평균만 있고 신뢰구간이 없다(summary.csv 에는 부트스트랩 95% 구간이 있다). 1% 차이를 논하는 라이브러리 교체 테이블 절에서 특히 필요하다.
- **W4. 시나리오가 합성 극단이다.** 1 개 긴 요청 + 31 개 짧은 요청, 무작위 토큰 프롬프트. 자연어 입력은 후속 실험에만 있고, 트레이스 기반 부하에서의 실생성은 없다. 4 장 다)의 빈도 분석과 라)의 이득이 같은 부하에서 연결되지 않는다.
- **W5. 출력 품질 지표가 없다.** 분기 사건 분류는 "왜 갈리는가"에 답하지만 "갈린 출력이 더 나쁜가"에는 답하지 않는다. 양자화·커널 최적화 논문은 보통 소규모 벤치마크 정확도(동일하거나 ±0.x)로 이를 보인다.
- **W6. 하드웨어 일반성 검증이 없다.** 단일 4090. 성능 모델의 SM 수·대역폭 what-if 는 예측일 뿐이며, Accel-Sim 은 미보정이다.

- **W7. vLLM 결과가 논지를 위협한다.** vLLM 0.31 에서 가변 길이 배치는 5.46 배 느리지만 백엔드와 무관했다. 이는 "운영 엔진에서는 attention 커널 선택보다 다른 비용이 지배한다"는 반론의 근거가 된다. 보고서는 이를 정직하게 적었지만, 프로파일링 결과가 없는 한 "본 작품의 이득이 운영 엔진에서 재현되는가"는 열린 질문으로 남는다.
- **W8. FlashInfer 비교의 버전 의존성.** tensor-core 변형이 가변 길이에서 느린 것이 0.6.13 의 특성일 수 있다. 최신 버전에서 한 번 더 격자를 돌리거나 버전 한정을 명시해야 한다.

4.3 보강 실험

- **V1. 정적 최선 분할 수 기준선(격자 분석은 GPU 불필요, 실생성은 GPU 반나절).** 격자 전체에서 "단일 고정 분할 수 s 의 셀별 손실"을 s=2~128 에 대해 계산하여 최악·p90 이 가장 작은 s 를 찾고(예상: 8 또는 16), 그 s 를 세 시나리오 + 64K 실생성에 넣어 TPOT 과 분기를 테이블·하이브리드와 나란히 보고한다. 결과가 어느 쪽이든 유익하다 — 고정 s 가 균일 길이에서 몇 % 손해를 보면 동적 선택의 정당화가 되고, 거의 같다면 "실용적 권고는 고정 16, 동적 선택은 긴 문맥·큰 배치에서"로 결론이 정리된다.
- **V2. 트레이스 구동 end-to-end(GPU, 1 일).** Azure 채팅 트레이스에서 200~500 개 요청 구간을 골라 도착 시각·길이를 그대로 엔진에 넣고(프롬프트는 길이를 맞춘 자연어), 휴리스틱·테이블·하이브리드의 TPOT 분포(p50·p90)와 처리량을 비교한다. 4 장 다)의 "13.5%, attention 시간 10.6% 절감"이 실측 TPOT 으로 얼마가 되는지 보인다. 재생기(traffic.py)가 배치 구성을 이미 만들어 주므로 시나리오 생성 비용이 작다.
- **V3. 자연어 벤치마크 정확도(GPU, 반나절).** GSM8K 또는 MMLU 일부(200~500 문항)를 가변 길이 배치로 묶어 greedy 생성하고 정확도를 휴리스틱 · 테이블 · 고정 16·FlashInfer CUDA-core 에서 비교한다. 정확도 차이가 0 이거나 ±0.x 이면 "출력 동일성" 절이 합성 프롬프트 밖에서도 성립한다는 근거가 된다. 또한 분기 사건의 빈도가 자연어에서 얼마나 낮은지(무작위 프롬프트 대비) 함께 보고한다.
- **V4. 신뢰구간 표기(GPU 불필요, 1 시간).** 표 8~10 의 배율에 부트스트랩 95% 구간을 괄호로 추가하고, 1% 미만 차이는 "구간이 겹침"으로 표기한다.
- **V5. SM 수 what-if 의 실측 검증(GPU, 반나절).** MPS 없이도 SM 일부를 점유하는 상주 커널(blocker kernel)로 유효 SM 수를 줄여 성능 모델의 SM 수 예측을 실측과 비교한다. 성능 모델이 "다른 GPU"에 대해 예측한 값의 신뢰도를 처음으로 확인하는 실험이며, 시뮬레이터 축의 약점(미보정)을 부분적으로 보완한다.
- **V6. vLLM 단계 프로파일링(GPU, 1 일).** torch profiler 로 디코드 단계 하나를 분해하여 attention 커널 시간과 그 밖의 시간(스케줄러, 블록 테이블 구성, 샘플러, 호스트 동기화)을 분리한다. attention 이 단계의 작은 비중이면 "운영 엔진에서는 다른 병목이 먼저"라는 결론을 적고, 큰 비중인데 분할 수가 1 이면 D6 의 이식이 정당화된다.
- **V7. FlashInfer 최신 버전 재측정(GPU, 1 시간).** 격자 162 셀을 최신 flashinfer-python 으로 다시 돌려 tensor-core 변형의 최악 손실이 유지되는지 본다. 유지되면 결론이 버전에 강건하고, 사라지면 "0.6.13 기준"으로 한정한다.

5. 파트 간 일관성에서 고칠 점 (보고서 문장 수준)

- **"출력 동일성"의 강도.** 요약·결론에서는 "생성 토큰을 유지하면서"라고 쓰고 4 장 라)(5)에서는 64K·FlashInfer 에서 갈렸다고 쓴다. 요약의 해당 문장은 "32K 조건에서 생성 토큰을 유지하면서"로

범위를 붙이는 편이 일관된다(v5 요약은 이미 "32K 조건의 … 분기 0 건"으로 적었으나 결론 첫 문단은 범위가 없다).

- **7,443 개 조건의 정의.** 6,801(워크로드×캐시 상태×후보)에 FlashInfer 422·64K 220 을 더한 값이다. 표 4 또는 4 장 가)에 이 분해를 적어 두어야 검증 항목과 맞춘다(현재 4 장 가)에 있음).
- **시뮬레이터의 역할**을 제목·요약·1 장에서 통일(W1 of §1).
- **"하이브리드 1.79 배"**는 테이블과 같은 선택을 한 결과이므로 "테이블과 같은 선택, 선택 비용만큼 느림"을 요약에도 한 구절 넣는 편이 과장 방지에 좋다.

6. 최종 보고서까지의 권장 순서

- V1(정적 기준선)과 E1-1(트레이스 채점 실측 검증)은 반나절씩이고 논리의 빈틈을 직접 메우므로 가장 먼저.
- V6(vLLM 프로파일링)은 결과에 따라 D6 의 필요 여부가 갈리므로 두 번째.
- V2(트레이스 구동 end-to-end)와 V3(벤치마크 정확도)는 최종 보고서의 평가 절을 "합성 극단"에서 "실제 부하·실제 입력"으로 옮기는 핵심이므로 세 번째.
- D4(선계산)·D1·D2(ablation)는 하이브리드·모델 정책을 유지할 경우에만. 정책을 테이블로 단순화하기로 하면 생략 가능.
- 관련 연구 문단 2 개(자동 튜닝·디스패치, 배치 불변성)는 GPU 없이 가능하므로 틈틈이.